

Competitive Programming

Section 63: AtCoder Difficulty Ladder

Competitive Programming Course

2026-07-24

Table of contents

1	Why this ladder is weighted toward hard tasks	3
2	Attempt protocol	4
3	Gate map	5
4	How to upsolve	6
5	Rating and language caveats	7
6	Exercises	8
6.1	A. Intermediate checkpoint	8
6.2	B. Reduction checkpoint	8
6.3	C. Red checkpoint	8
7	Submission checklist	9

1 Why this ladder is weighted toward hard tasks

The first five tasks establish AtCoder conventions; they are not the center of the section. Rungs 6–30 demand reliable recognition and implementation, 31–39 combine several course tools, and 40–50 move through orange into red-level reductions. Difficulty is nondecreasing by rounded community rating.

2 Attempt protocol

Before reading an editorial, write:

```
slow exact model  
state or invariant  
first complexity bottleneck  
small hostile example
```

At the orange/red gates, a template name is not a model. Derive what every state, edge, coefficient, or implication means in the original problem.

3 Gate map

Gate	Rungs	Main pressure
1	1–5	exact loops, windows, functional graphs, tree propagation
2	6–15	graph basics, subset DP, heap greedy, offline queries
3	16–30	compressed DP, algebraic transforms, flow, range structures
4	31–39	quotient compression, geometry, Mo, lazy composition, cuts
5	40–45	proof-heavy string/expectation/tree reductions
6	46–50	frontier DP, 2-SAT compression, NTT, subset transforms

4 How to upsolve

Read only through the first missing observation. Close the editorial and reconstruct the algorithm, proof, and complexity from memory. Then preserve the smallest test that disproved your original model. For red tasks, also implement a slow oracle on tiny inputs before the optimized version.

5 Rating and language caveats

AtCoder task colors are community conventions derived from estimated difficulty. Estimates move as more submissions are observed. The official English task page controls the contract; the offline sheet is a compact practice brief rather than a replacement.

6 Exercises

6.1 A. Intermediate checkpoint

Complete rungs 6–15. For each task, record one tempting wrong model and the smallest input that disproves it.

6.2 B. Reduction checkpoint

For rungs 28, 33, 35, and 39, draw the graph, range structure, or state space used by the solution and label what every object means in the original statement.

6.3 C. Red checkpoint

For each of rungs 46–50, first implement or specify a tiny exact oracle. Use it to test the optimized reduction on exhaustive small instances.

7 Submission checklist

- ☐ I read the official contract before submitting.
- ☐ I can state the invariant without the story.
- ☐ I proved the reduction used by my data structure.
- ☐ My complexity fits the constrained quantity.
- ☐ I kept one hostile regression test.